

Workflow Management mit Nextflow (Part 2)

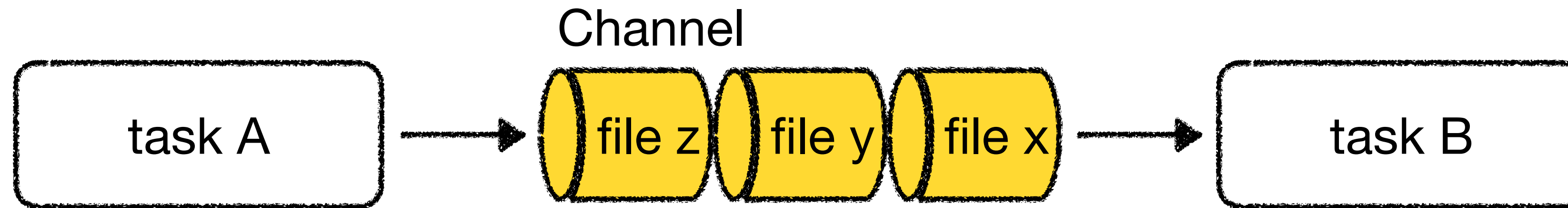
Ioana Lemnian, PathoNext GmbH
ioana.lemnian@pathonext.de

What is Nextflow?

- Based on the **dataflow** programming paradigm
- Tasks are defined as **processes** (isolated units of computation written in any scripting language)
- Processes communicate with each other through **channels** (asynchronous FIFO queues that can send and receive any kind of data)

Channels

- Key data structure of Nextflow, used to logically connect tasks to each other
- Main way of communicating between processes in a pipeline



- Channel types:
 - queue channels
 - value channels

Queue channel

- implicitly created by process output definitions or using channel factories such as `Channel.of`, `Channel.fromPath`, `Channel.fromFilePairs`
- `view` channel operator prints each item emitted by the channels

```
workflow{  
  ch = Channel.of('Hello', 'Ciao', 'Hallo', 'Salut')  
  ch.view()  
}
```

Value channel

- bound to a single value
- it can be read unlimited times without consuming its contents
- it can be created using the `value` channel factory or by operators returning a single value like `first`, `last`, `collect`, `count`, `min`, `max`, `sum`

```
expl1 = Channel.value()  
expl2 = Channel.value( 'Hello there' )  
expl3 = Channel.value( [1,2,3,4,5] )
```

Operators

- Methods that allow to connect channels, transform values emitted by channels, or apply some user provided rules
- Basic operators:
 - `view()` - prints the items emitted by a channel to the console standard output, each item in a new line
 - `map()` - applies a function to every item emitted by a channel and returns the items obtained as a new channel.
 - `mix()` - combines the items emitted by two or more channels into a single channel
 - `collect()` - collects all the items emitted by a channel in list and returns the object as a sole emission
 - `join()` - join of the items emitted by two channels with a matching key (first element in each item)

Process

- basic computing unit to execute tasks like custom scripts or tools
- process definition starts with the keyword `process`, followed by the process name and finally by the process body delimited by curly brackets
- Convention: `process` name is commonly written in upper case

Process

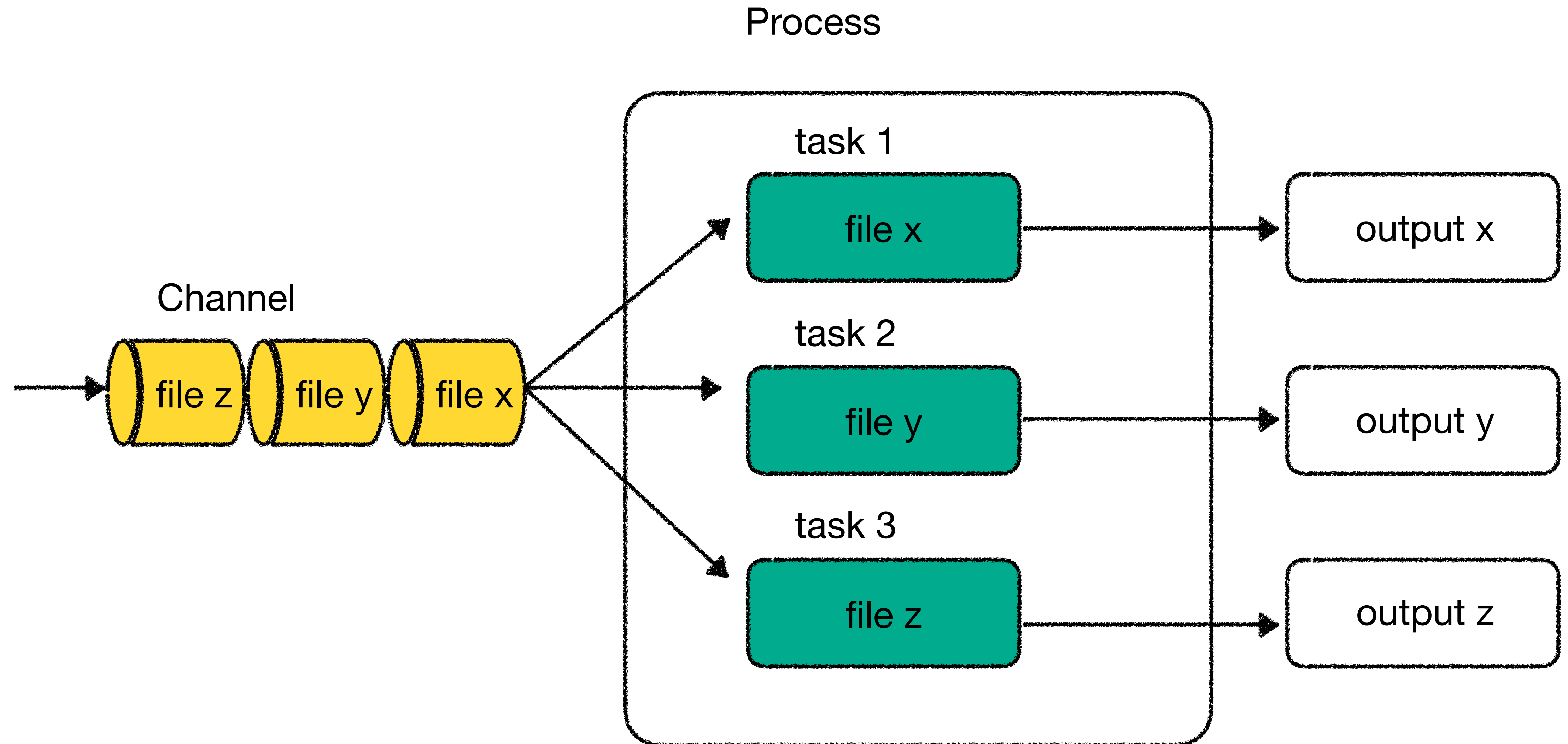
- **Process body** can contain up to 5 definition blocks:
 - **directives** - initial declarations that define optional settings
 - **input** - expected input channels
 - **output** - expected output channels
 - **when** - optional clause statement to allow conditional processes
 - **script** - command to be executed

```
process < name > {  
    [ directives ]  
  
    input:  
    < process inputs >  
  
    output:  
    < process outputs >  
  
    when:  
    < condition >  
  
    [script|shell|exec]:  
    ""  
    < user script to be executed >  
    ""  
}
```

Directives

- allow the definition of optional settings that affect the execution of the current process without affecting the semantic on the task itself.
- define the amount of computing resources to be used or meta directives:
 - `cpus` - number of CPUs required by the task
 - `time` - how long is the task allowed to run
 - `memory` - how much memory is the task allowed to use
 - `disk` - how much local disk storage is the task allowed to use
 - `tag` - label each process execution in order to identify it easier in the log file
 - `cache` - allows resuming the execution from the last successfully executed step
 - `publishDir` - result folder

Input / Output



Input/Output

- process instances are isolated from each other but can communicate between themselves by sending values through channels
- inputs determine the dependencies and the parallel execution of the process
- inputs can be combined
- inputs/outputs can be for example:
 - `val` - value of any type
 - `path` - file values
 - tuples of values

Tuple of values as input/output

```
reads_ch = Channel.fromFilePairs('data/*_R{1,2}.fq')

process F00 {
  input:
  tuple val(sample_id), path(sample_id_paths)

  output:
  tuple val(sample_id), path('sample.bam')

  script:
  """
  echo your_command_here --reads $sample_id_paths > sample.bam
  """
}

workflow {
  bam_ch = F00(reads_ch)
  bam_ch.view()
}
```

When

- set condition that must be verified in order to execute the process.

```
params.dbtype = 'nr'
params.prot = 'data/prot/*.tfa'
proteins = Channel.fromPath(params.prot)

process FIND {
  debug true

  input:
  path fasta
  val type

  when:
  fasta.name =~ /^BB11.* / && type == 'nr'

  script:
  """
  echo blastp -query $fasta -db nr
  """
}

workflow {
  result = FIND(proteins, params.dbtype)
}
```

Script

- by default, the process command is interpreted as a Bash script
- can be changed with the corresponding Shebang declaration

```
process PYSTUFF {  
      
      
    #!/usr/bin/env python  
  
    x = 'Hello'  
    y = 'world!'  
    print ("%s - %s" % (x, y))  
      
}  
  
workflow {  
    PYSTUFF()  
}
```

Script parameters

- can be defined dynamically using variable values

```
params.data = 'World'

process F00 {
  script:
  """
  echo Hello $params.data
  """
}

workflow {
  F00()
}
```

Nextflow configuration

- when a workflow script is launched, Nextflow looks for a file names `nextflow.config` in the current directory, in the script base directory, and finally in `$HOME/.nextflow/config`
- Information to store in the config file:
 - process directives (good practice to decouple the workflow execution logic from the process configuration settings)
 - needed container image (Docker/Singularity)
 - conda environment